

# frontier2d\_sparse: $\theta$ マスク疎評価による 価値反復の高速化

ソルバ仕様ノート

2026-06

## 概要

`frontier2d_sparse` は、価値反復 (VI) の状態ごとの更新式を一切変えずに計算を速くする CPU ソルバである。原論文の brute-force 価値反復は、全状態を走査してすべての状態の最適コストを厳密に求める。これは大きな強みだが、地図が横長になるほどスweep回数が増え、計算量も増える。本ソルバはこの厳密解を保ったまま、(i) 値が変わり得る状態だけを追う**活性集合 (フロンティア)**、(ii) ペナルティを値へ畳み込んだ**融合レイアウト  $cp$** 、(iii) **非同期 Gauss–Seidel 並列**、(iv)  $\theta$  **マスク疎評価**、の四つの工夫で計算量を削る。とくに (iv) は、向き  $\theta$  方向の依存が疎であることを使い、前スweepに変化した  $\theta$  だけを再評価して冗長な計算を省くものである。本ノートは各工夫を数式と図で示し、いずれもが brute-force 価値反復と**同じ解** (到達可能な状態の値・方策が 64 ビット単位で一致) を与えることを証明する。記法・変数記号は Ueda ら (2023) の brute-force value iteration<sup>\*1</sup> に合わせる。

## 目次

|   |                          |   |
|---|--------------------------|---|
| 1 | 準備：価値反復と Bellman 更新      | 1 |
| 2 | 工夫 1：活性集合 (フロンティア)       | 2 |
| 3 | 工夫 2：ペナルティ融合レイアウト $cp$   | 3 |
| 4 | 工夫 3：非同期 Gauss–Seidel 並列 | 4 |
| 5 | 工夫 4： $\theta$ マスク疎評価    | 4 |
| 6 | 正しさ：全走査の解と一致する           | 6 |
| 7 | まとめ                      | 6 |

## 1 準備：価値反復と Bellman 更新

高速化を述べる前に、対象となる価値反復の問題と記法を定義する。

---

<sup>\*1</sup> R. Ueda, L. Tonouchi, T. Ikebe, and Y. Hayashibara, “Implementation of Brute-Force Value Iteration for Mobile Robot Path Planning and Obstacle Bypassing,” *J. Robot. Mechatron.*, Vol.35, No.6, pp. 1489–1502, 2023.

■**状態と定数**。格子は  $N_x \times N_y \times N_\theta$ 、状態は  $s = (i_x, i_y, i_\theta)$  (平面位置と向き)、状態数は  $N_s = N_x N_y N_\theta$ 。平面の 1 マスをセル  $c = (i_x, i_y)$  と呼ぶ。値とコストは 64 ビット整数 (u64) 上の固定小数点で表し、下位 18 ビットが小数部、整数  $B = 2^{18}$  が実数 1.0 (= 1 歩のコスト) にあたる。未到達・占有を表す番兵は原論文と同じく  $R_{\max} = 10^9 B$ 。各状態は値 (cost-to-go)  $V(s)$  と二つのペナルティ  $\rho(s), \lambda(s)$  を持つ。桁あふれを  $2^{64}$  の余りに丸める加算・減算・乗算 (wrapping 演算) を田, 曰, 冏、整数右シフトを  $\gg b_B$  ( $b_B = 18$ ) と書く。自由 (通行可能) な状態の集合を  $\mathcal{F}$ 、吸収ゴールの集合を  $\mathcal{X}_{\text{goal}} \subseteq \mathcal{F}$  とし、1 歩あたり必ずかかるステップコスト (原論文 Eq. (3) の  $r(s')$ ) を  $r(s) = \rho(s) \text{ 田 } \lambda(s) \geq B > 0$  と置く。

■**状態遷移確率**。同じ行動でも源セル内の位置で着地先がばらつくため、各セルを  $K^3$  個の小領域に分けて運動モデルで飛ばすモンテカルロ積分で、相対変位  $\Delta s'$  の状態遷移確率を得る。これを整数の重み付き分布

$$\{(\Delta s', p)\} = P(\Delta s' | i_\theta, a), \quad \Delta s' = (d_{ix}, d_{iy}, d_{it}), \quad \sum p = B \quad (1)$$

として持つ (確率は  $P = p/B$ 、 $d_{it}$  は絶対の角度インデックス)。状態  $s$  から相対変位  $\Delta s'$  で移った先は

$$s \oplus \Delta s' = (i_x + d_{ix}, i_y + d_{iy}, d_{it} \bmod N_\theta). \quad (2)$$

■**Bellman 更新**。行動  $a$  のコストは遷移先の「値 + ステップコスト」を確率で平均したもので、各状態ではそれを最小にする行動を選ぶ。これが原論文 Eq. (13) の更新で、固定小数点では次のように実体化される (確率  $P = p/B$  ゆえ総和を  $B$  で割る  $\gg b_B$  が入る。遷移先が通行不可／範囲外なら番兵  $R_{\max}$ ) :

$$A(s, a) = \left( \bigoplus_{(\Delta s', p)} [V(s \oplus \Delta s') \text{ 田 } r(s \oplus \Delta s')] \text{ 冏 } p \right) \gg b_B, \quad (3)$$

$$(\mathcal{T}V)(s) = \min_{a \in \mathcal{A}} A(s, a), \quad a^* = \pi(s) = \arg \min_{a \in \mathcal{A}} A(s, a) \quad (s \in \mathcal{F} \setminus \mathcal{X}_{\text{goal}}). \quad (4)$$

通行不可セルとゴールでは更新しない (ゴールは  $V = 0$  に固定)。価値反復とは、この固定点  $V^* = \mathcal{T}V^*$  を求めることである。 $r > 0$  かつゴールへ確率 1 で到達できることから、これは割引なし確率的最短経路の問題となり、次が成り立つ。

**命題 1** (到達可能な状態で固定点は一意)。固定点  $V^*$  は到達可能な状態 ( $V < 10^6 B$ ) の上で一意に定まる。

この一意性が本ソルバの設計を支える。 $\mathcal{T}$  を**状態単位で同じ式**で適用する限り、どの状態をどんな順序・並列度で更新するかは**速度だけ**を左右し、到達可能な状態の値・方策は変わらない。以下の四つの工夫はすべてこの事実の上に立つ。

■**依存の半径**。ある状態の値が変わったとき、どこまで遠くの状態を再計算すべきかを決めるため、全遷移の最大変位

$$m_x = \max_{a, t, \Delta s'} |d_{ix}|, \quad m_y = \max_{a, t, \Delta s'} |d_{iy}|, \quad m_t = \max_{a, t, \Delta s'} \text{circ}(d_{it} - t) \quad (5)$$

を定める。 $\text{circ}(u) = \min(u \bmod N_\theta, N_\theta - (u \bmod N_\theta))$  は角度を円環とみた距離である。 $(m_x, m_y)$  は空間方向、 $m_t$  は源の向きからの角度方向の依存半径を表す (§5 で使う)。

## 2 工夫 1：活性集合（フロンティア）

■狙い。 brute-force な全走査は、すべての状態を毎スweep更新することで厳密性を保証する。一方、値はゴールから 1 歩ずつしか広がらないため、1 スweepで実際に値が変わるのは、確定した領域と未確定の領域の境目（**波面**）にある状態に限られる（図 1）。そこで厳密解はそのままに、値が変わった状態とその近傍だけを次の更新対象に絞り、 $O(\text{diam} \cdot N_s)$  の計算量を削る。

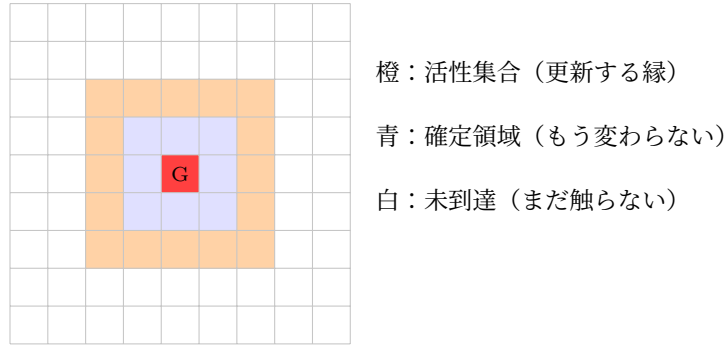


図 1 活性集合（フロンティア）。確定領域（青）の縁にある状態（橙）だけを更新し、外側の未到達セル（白）には触れない。スweepごとに橙の輪が外へ広がる。

■定義。 スweep  $k$  のフロンティア  $F_k$  を「直前のスweepでいずれかの向き  $\theta$  の値が下がったセル」とする。初期値は  $F_0 = \{c : \exists i_\theta, V(c, i_\theta) < R_{\max}\}$ （ゴールを含むセル）。各スweepの候補は、フロンティアを依存半径  $(m_x, m_y)$  だけ空間膨張させた集合

$$C_k = \text{dil}_{(m_x, m_y)}(F_{k-1}) = \{c : \exists c' \in F_{k-1}, |i_x - i'_x| \leq m_x, |i_y - i'_y| \leq m_y\} \quad (6)$$

とする。候補  $C_k$  の各セルで全  $\theta$  層に  $\mathcal{T}$  を適用し、値が下がった  $\theta$  を持つセルを集めて次のフロンティア  $F_k$  とする。 $F_k$  が空になれば収束である。

**命題 2**（候補集合は取りこぼさない）。直前のスweepの変化に依存して次に値が変わり得る状態は、すべて候補  $C_k$  (6) に含まれる。

**理由。** ある状態  $s$  の値に影響する手前の状態  $s'$  は  $s = s' \oplus \Delta s'$  を満たす、すなわち  $s$  から空間変位  $(-d_{ix}, -d_{iy})$  以内にある。依存半径  $(m_x, m_y)$  で膨らませればそれらをすべて覆う。向き  $\theta$  方向は候補セル内で全層を評価するのでここでは漏れない。ゆえに  $C_k$  は真に再計算が要る状態の上位集合であり、活性集合の収束値は全走査と一致する。□

## 3 工夫 2：ペナルティ融合レイアウト $cp$

■狙い。 コスト (3) は遷移先ごとに値  $V$  とペナルティ  $\rho, \lambda$  を別々に読む。フロンティア系の内側ループはメモリの読み出し量に律速されるので、読む量を減らせば速くなる（図 2）。

■融合。 各セルについて、値とステップコストをあらかじめ足した一語

$$cp(s) = V(s) \boxplus r(s) = V(s) \boxplus \rho(s) \boxplus \lambda(s) \pmod{2^{64}}, \quad \text{pen}(s) = \rho(s) \boxplus \lambda(s) = r(s) \quad (7)$$

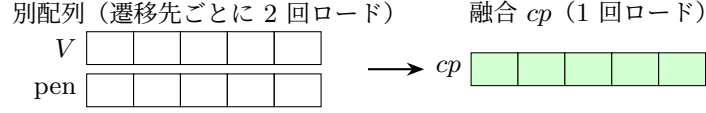


図2 ペナルティ融合。値とステップコストを一語  $cp = V \boxplus r (= V \boxplus \rho \boxplus \lambda)$  に畳み込むと、遷移先ごとのロードが2回から1回になり、帯域律速が緩む。

を持たせる。 $cp$  は8バイトの単一読み出しで済み、 $pen$  は自セルの値を戻すときだけ使う。通行不可・未確定 ( $V = R_{\max}$ ) のセルには番兵  $cp = \text{UNREACHED} = 2^{64} - 1$  を入れる (実セルの  $cp$  はこれより十分小さく、衝突しない)。コストは

$$A(s, a) = \left( \bigoplus_{(\Delta s', p)} cp(s \oplus \Delta s') \boxdot p \right) \gg b_B \quad (\text{いずれかの } cp = \text{UNREACHED} \text{ なら } R_{\max}) \quad (8)$$

と書け、遷移先ごとの読み出しが17バイトから8バイトへ減り、通行可否の分岐と  $r$  の加算も消える。

**補題 1** (融合しても結果は同じ). (8) は (3) とすべての  $(s, a)$  で完全に一致する。

**理由.**  $2^{64}$  法の加算は順序を変えても結果が同じなので、 $V \boxplus r$  を先にまとめても総和は変わらない。番兵への分岐は元の「通行不可・未確定なら  $R_{\max}$ 」と同じ条件・同じ位置で同じ値を返す。自セルの現在値は  $V(s) = cp(s) \boxdot pen(s)$  で正しく戻せる (足し引きは可逆だから)。□

■はみ出しの除去。地図を依存半径  $(m_x, m_y)$  ぶん通行不可セルで囲っておくと、範囲外への参照は自動的に番兵を読んで  $R_{\max}$  になり、境界判定の分岐が不要になる。また、絶対角度と配列の並びから、遷移先の位置は基準列+一定のずれで表せるので、 $(a, \text{源の向き})$  ごとにその「ずれと確率」を前計算しておけば、内側ループから掛け算と余り計算を消せる。これらも計算順序を変えないので結果は同じである。

## 4 工夫3：非同期 Gauss–Seidel 並列

候補  $C_k$  を  $m$  本のスレッドで分担する。各スレッドは共有カウンタでブロック (16 セル) を取り合い (work stealing)、担当範囲の  $cp$  をその場で書き換える。ブロック単位で取り合うので、1つのセルを書くのは常に1スレッドだけである。スイープの境目は二つのバリアで区切り、走査方向をスイープごとに反転して収束を均す。

**命題 3** (非同期並列でも正しく収束する). 値は減る一方で、真の固定点  $V^*$  が下限であり、各セルの書き手は1つだけである。よって反復は必ず止まり、「1スイープ丸ごと変化なし」で止まったときは全状態が (4) を満たす。これは一意固定点  $V^*$  であり、到達可能な状態の値・方策は全走査の解と一致する。

他スレッドが書きかけの値を読むこともある (逐次更新と一括更新の中間) が、 $A(s, a)$  は  $cp$  について単調で、固定点ではすべての状態が同時に整合するため、読む順序は**最終結果**を変えない。

## 5 工夫 4： $\theta$ マスク疎評価

■冗長な再評価の所在。工夫 1 の候補セルは、全  $N_\theta$  個の向きを毎スweep再評価していた。実測（ $384 \times 384 \times 60$  の実地図）では、全  $\theta$  を評価する版は Bellman 評価 52.8M 回に対し実際に値が下がったのは 16.9M 回（約 1/3）で、残り 2/3 は値が変わらない再評価だった。これは正しさには寄与するが省ける計算であり、原因は向き  $\theta$  方向の依存が疎であることにある。

■向き方向の依存。状態  $(c, \theta)$  のコスト (8) の入力は遷移先  $(c + \Delta_{xy}, \theta')$  であり、その  $\theta'$  は源の  $\theta$  から角度距離  $\text{circ}(\theta' - \theta) \leq m_t$  の範囲にしか現れない（回転行動は  $d_{ix} = d_{iy} = 0$  なので自セルも入力に入る）。これを裏返すと、次が言える。

補題 2（向きの局所性）．前スweepにセル  $n$  で値が下がった向きの集合を  $\Theta_n$  とすると、その変化に依存して再評価が要るのは、 $c$  が  $n$  の空間窓  $(\pm m_x, \pm m_y)$  内にあり、かつ

$$\theta \in \text{rotdil}(\Theta_n, m_t) := \bigcup_{j=-m_t}^{m_t} \text{rot}_j(\Theta_n) \quad (9)$$

を満たす  $(c, \theta)$  に限られる。 $\text{rot}_j$  は周期  $N_\theta$  の円環シフト、 $\text{rotdil}(\cdot, m_t)$  はそれを  $\pm m_t$  ぶん広げた円環膨張である（図 3 左）。

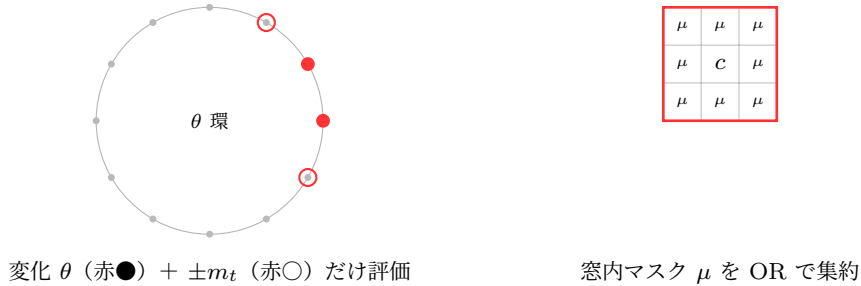


図 3  $\theta$  マスク疎評価。左：向きを円環とみて、前スweepに変化した向き（赤●）とその  $\pm m_t$ （赤○）だけを再評価し、 $N_\theta$  全部は評価しない。右：空間窓  $(\pm m_x, \pm m_y)$  内のセルの変化マスク  $\mu$  をまとめて集約する。

■変化マスクの利用。そこで各セルに、「直前のスweepで値が下がった向きビット」を表すマスク  $\mu(c) \in \{0, 1\}^{N_\theta}$  を持たせる（ $N_\theta \leq 64$  なので u64 一語に収まる）。候補セル  $c$  を評価するときは、空間窓内のマスクをまとめて（OR で）集め、1 回だけ円環膨張する（図 3 右）：

$$\text{evalmask}(c) = \text{rotdil}\left(\bigcup_{|u| \leq m_x, |v| \leq m_y} \mu(c + (u, v)), m_t\right). \quad (10)$$

$\text{evalmask}(c) = 0$  なら入力に変化がないのでセルごと飛ばす。そうでなければ、立っている向きビットだけに (8)–(4) を評価する。マスク配列は約 1MB でキャッシュに常駐するため、この集約は遷移先を 1 つずつ読むより十分安い。

マスクの整合はスweep境界の直列区間で保つ：前スweepに書いたマスクをゼロに戻し、今スweepで値が下がったセルのマスクを書き込む（どちらも変化セル数に比例した手間）。セルの書き

手は 1 つなので、上書きで足りる（排他制御は不要）。最初のスweepはマスクが育っていないので全  $\theta$  を評価するが、これは上位集合なので正しさを損なわない。

#### 1 スweepの処理（並列、セル単位で取り合い）：

各候補セル  $c$  について：

初回スweep：  $\text{emask} = \text{全 } \theta$

それ以外：

$\text{acc} =$  窓内  $|u| \leq m_x, |v| \leq m_y$  のマスク  $\text{mu}[c+(u,v)]$  を OR で集約

$\text{acc} == 0$  なら  $c$  を飛ばす # 入力に変化なし

$\text{emask} = \text{rotdil}(\text{acc}, \text{mt})$  # 円環  $\pm \text{mt}$  膨張

$\text{cmask} = 0$

$\text{emask}$  の立った向き  $\theta$  について：

評価対象でなければ飛ばす

$\text{before} = \text{cp}[c, \theta] - \text{pen}[c, \theta]$  # 自値を戻す

$q = \min \text{ over } a \text{ of } A((c, \theta), a)$  # 式(8) のコスト

$q < \text{before}$  なら：

$\text{cp}[c, \theta] = q + \text{pen}[c, \theta]$ ;  $\text{cmask}$  に  $\theta$  を立てる

$\text{cmask} != 0$  なら 変化リストに  $(c, \text{cmask})$  を追加

バリア（全書き込みを可視化）

リーダー（直列）：

前スweepのマスクをゼロ化 → 変化リストから  $\text{mu}[c] = \text{cmask}$  を書く → 次フロンティア  $F_k$  を作る

$F_k$  が空、または反復上限なら終了

バリア（次の候補・マスクを可視化）

## 6 正しさ：全走査の解と一致する

**定理 1** ( $\theta$  疎評価でも解は変わらない). *frontier2d\_sparse* が「1 スweep丸ごと変化なし」で止まったとき、到達可能な状態の値  $V$  と方策  $\pi$  は、全状態を走査する *brute-force* 価値反復の解と 64 ビット単位で完全に一致する。

証明. 評価する集合 (10) は、真に依存する状態の取りこぼしのない上位集合である：空間方向は  $(\pm m_x, \pm m_y)$  窓 (命題 2)、向き方向は円環  $\pm m_t$  膨張 (補題 2) で、コスト (8) の真の入力  $(c + \Delta_{xy}, \theta')$  を必ず覆う。よって、入力が変わった  $(c, \theta)$  は必ず次スweepの評価対象に入る。したがって、どの候補でも変化がなくなって止まったとき、すべての  $(c, \theta)$  は現在値で Bellman 式 (4) を満たしている。これは一意固定点  $V^*$  (命題 1) の定義そのものである。融合 (補題 1) と非同期並列 (命題 3) も解を変えないので、到達可能な状態の値・方策は全走査の解に一致する。□ □

この一致は回帰テストで確認できる。空き地図・障害物入り地図・ゴールを囲んだ地図、および大規模の空き地図で、全走査の固定点と**完全に一致**（不一致 0）することを検証している。

## 7 まとめ

`frontier2d_sparse` は、Bellman 更新 (4) を変えずに、活性集合 (§2) で直径律速を、融合レイアウト  $cp$  (§3) と非同期並列 (§4) で読み出し量と並列度を、 $\theta$  マスク疎評価 (§5) で冗長な向きの再評価を、それぞれ削る。どの工夫も「真に依存する状態を取りこぼさない上位集合だけを評価する」点で正当化され、止まったときに一意固定点へ達するため、到達可能な状態の値・方策は全走査の解と完全に一致する (定理 1)。